Programming in Python
IDC 212

Lecture - 5

August 26, 2025

Contents

1. Modules

2. Packages

3. Class and Objects

Modules

What is a Python Module?

- A **module** is a file containing Python definitions and statements.
- Every file with a `.py` extension is treated as a module.
- Modules allow us to organize code logically and reuse it.
- A module may contain:
  - Functions
  - Variables
  - Classes

Creating a Module: mathutils.py

mathutils.py

```python
num = 100

def add(a, b):
    print("Sum is:", a + b)

def multiply(a, b):
    print("Product is:", a * b)
```

- The module `mathutils` defines:
  - a variable: `num`
  - two functions: `add`, `multiply`

Using the Module in Another Script

test.py

```python
import mathutils

print(mathutils.num)
mathutils.add(10, 15)
mathutils.multiply(10, 15)
```

Output:

```
100
Sum is: 25
Product is: 150
```

Accessing Members of a Module

- We access members using the module name:
  - `mathutils.num`
  - `mathutils.add()`
  - `mathutils.multiply()`
- This keeps our code organized and modular.

Compiled Bytecode for Modules

- When a module is imported for the first time, Python compiles it into bytecode.
- The compiled file is stored as `__pycache__/mathutils.cpython-...pyc`.
- This makes future imports faster.
- You don't need to manage this file manually.

Renaming a Module at the Time of Import (Module Aliasing)

- **Eg:** `import mathutils as m`
- Here `mathutils` is the original module name and `m` is the alias.
- We can access members using the alias.

Module Aliasing: Example

test.py

```python
import mathutils as m

print(m.x)
m.add(10, 20)
m.multiply(10, 20)
```

Using `from < > import < >`

We can import specific members directly.

Advantage: No need to prefix with module name.

from < > import < > Example

```python
from mathutils import x, add

print(x)
add(10, 20)
multiply(10, 20)  # Error: multiply not defined
```

Error: `NameError: name 'multiply' is not defined`

Importing All Members from Module

You can import everything using `from module import *`.

`from < > import *` Example

```python
from mathutils import *

print(x)
add(10, 20)
multiply(10, 20)
```

Note: Will work only if `multiply` is defined in `mathutils`.

Various Possibilities of Import

1. import module_name
2. import module1, module2, module3
3. import module1 as m
4. import module1 as m1, module2 as m2
5. from module import function
6. from module import f1, f2, f3
7. from module import f1 as alias
8. from module import *

Member Aliasing in Python

What is Member Aliasing?
Sometimes we may want to rename specific members while importing them from a module. This helps avoid conflicts or simplify usage.

Example:

```python
# mathutils.py contains x = 50, add(a,b), product(a,b)

from mathutils import x as val, add as addition
print(val)
addition(5, 10)
```

Alias Usage Rule

Once we define an alias name, we should use only the alias name in the program.

Incorrect Usage:

```
from mathutils import x as val
print(x)  # NameError: name 'x' is not defined
```

Explanation: The original name ‘x‘ is no longer available in the local scope.

Finding Members using `dir()`

Python provides a built-in function `dir()` to list all members of a module.

Syntax:

- `dir()` – Members of the current module
- `dir(moduleName)` – Members of a specific module

Example:

```
x = 10
def greet():
    print("Hello")

print(dir())  # shows all members like 'x', 'greet', '__name__' etc.
```

Using dir() on a Module

mathutils.py

```python
x = 50

def add(a, b):
    print("Sum:", a + b)

def product(a, b):
    print("Product:", a * b)
```

test.py

```python
import mathutils
print(dir(mathutils))
```

Output of dir(mathutils)

When we use dir(mathutils), the result includes user-defined items and special internal dunder variables.

Output Example:

```
['__builtins__', '__cached__', '__doc__', '__file__',
 '__loader__', '__name__', '__package__', '__spec__',
 'add', 'product', 'x']
```

These are all members currently in the module ‘mathutils‘.

What Are These Special Dunder Members?

These double underscore variables are automatically added by the Python interpreter at runtime.

Common Dunder Attributes:

- `__builtins__`
- `__cached__`
- `__doc__`
- `__file__`
- `__loader__`
- `__name__`
- `__package__`
- `__spec__`

Purpose: These store metadata about the module and help Python manage import behavior.

The Special Variable `__name__`

- Every Python file has a built-in variable called `__name__`.
- It indicates whether the file is run as a script or imported as a module.
- If run directly, `__name__` is set to `"__main__"`.
- If imported, it takes the name of the module (i.e., the filename without extension).

Purpose of __name__ Variable

- Helps to differentiate between code that should run when:
  - the file is executed as the main program
  - the file is imported from another module
- Enables conditional execution using:

```
if __name__ == "__main__":
    # block to run only if script executed directly
```

Demo Code in utils.py

```python
def greet():
    if __name__ == "__main__":
        print("Running as a standalone script")
    else:
        print("Imported as a module into another script")

greet()
```

Calling `utils.py` Directly from Terminal

Command:

```
$ python utils.py
```

Output:

```
Running as a standalone script
```

Importing utils in Another Script

Code: `main.py`

```python
import utils
utils.greet()
```

Output:

```
Imported as a module into another script
```

Summary: `__name__` Use Case

- Use `if __name__ == "__main__"` to write test or demo code.
- Keeps reusable code clean and testable.
- Avoids executing certain blocks when imported.

Best Practice:

```python
def main():
    # your script logic here

if __name__ == "__main__":
    main()
```

Accessing __doc__ of a Function

- The __doc__ attribute returns the docstring of a function, class, or module.
- This is useful for documentation and runtime inspection.

Example:

```python
def greet(name):
    """This function greets the person with the given name."""
    print("Hello", name)

print(greet.__doc__)
```

Output:

```
This function greets the person with the given name.
```

Packages

Introduction to Python Packages

- A package is a directory that contains related Python modules.
- Any directory with an `__init__.py` file is considered a package.
- Packages allow hierarchical organization of modules.
- Subpackages are packages nested inside another package.

Basic Package Structure

Example Structure:

```
BankServices/
   __init__.py
   main.py
   HomeLoan/
      __init__.py
      home.py
      utils.py
   VehicleLoan/
      __init__.py
      vehicle.py
      helpers.py
```

Why Use Packages?

Advantages of Python Packages:

1. They provide encapsulation and modularity.
2. Help avoid naming conflicts.
3. Enable better code organization in large projects.
4. Allow hierarchical import of modules.
5. Support reusable components across projects.

Accessing Modules in Subpackages

Example Usage:

```python
from BankServices.HomeLoan import home
from BankServices.VehicleLoan import vehicle

home.apply_loan()
vehicle.calculate_emi()
```

Explanation:

- Submodules are imported using dotted path notation.
- The package hierarchy is preserved in imports.

Summary

- A Python package is a collection of modules grouped in directories.
- Subpackages allow hierarchical structure.
- The `__init__.py` file is essential for packages.
- Packages improve maintainability and scalability of Python code.

Importing Everything from a Package

- In Python, `from packagename import *` imports only the symbols listed in the `__all__` list of the package's `__init__.py` file.
- This allows you to control what is accessible when someone uses the wildcard import.
- If `__all__` is not defined, nothing is imported by default using `*`.

Example: Defining __all__ in a Package

Directory Structure:

```
mathutils/
    __init__.py
    adder.py
    multiplier.py
```

__init__.py

```
__all__ = ['adder']
```

adder.py

```
def add(x, y):
    return x + y
```

multiplier.py

```
def multiply(x, y):
    return x * y
```

Using from mathutils import *

main.py

```python
from mathutils import *

print(add(3, 4))        # Works
print(multiply(3, 4))   # NameError: multiply not defined
```

- Only ‘add‘ is imported because of ‘__all__ = [’adder’]‘.
- To access ‘multiply‘, import ‘multiplier‘ explicitly.

Using from mathutils import *

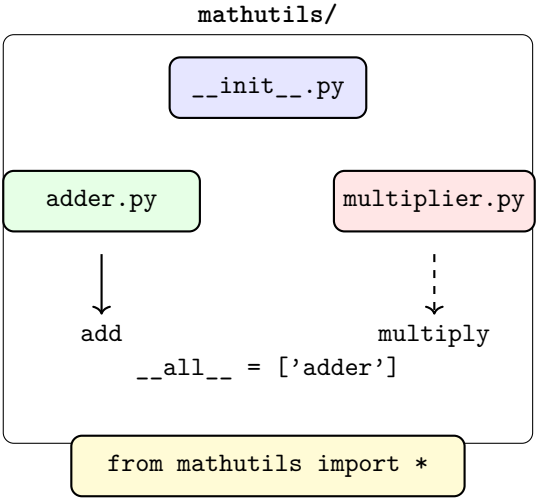
main.py

```python
from mathutils import *

print(add(3, 4))        # OK: works
print(multiply(3, 4))   # Error: multiply not defined
```

- Only add is imported because of `__all__ = ['adder']`.
- To access `multiply`, import `multiplier` explicitly.

Package Import with *



Class and Objects

What is Class in Python?

- Python is an object-oriented language.
- A class is a blueprint for creating objects.
- A class can include:
  - **Variables**: Data (fields)
  - **Methods**: Functions to manipulate data

Defining a Class in Python

Syntax:

```python
class ClassName:
    """This is a docstring."""
    # variables and methods
```

Example:

```python
class Student:
    """Represents student details"""
    pass
```

Accessing Class Documentation

```python
print(Student.__doc__)    # Prints docstring
help(Student)             # Displays help message
```

Types of Variables in Python Class

- **Instance Variables:** Defined using `self` inside methods.
- **Class Variables:** Shared by all instances; defined at class level.
- **Local Variables:** Declared inside a method; used temporarily.

Types of Methods in Python Class

- **Instance Method:** Takes `self` as first argument.
- **Class Method:** Takes `cls` as first argument; use `@classmethod`.
- **Static Method:** Doesn't take `self` or `cls`; use `@staticmethod`.

Example with All Method Types

```python
class Math:
    pi = 3.14

    def __init__(self, r):
        self.radius = r

    def area(self):
        return Math.pi * self.radius ** 2

    @classmethod
    def unit_circle(cls):
        return cls(1)

    @staticmethod
    def greet():
        print("Welcome to Math class!")
```

Using the Methods

```python
m1 = Math(2)
print(m1.area())          # Instance method

m2 = Math.unit_circle() # Class method
print(m2.radius)

Math.greet()              # Static method
```

Class Definition in Python

Example: Simple Python Class

```python
class Student:
    """Python demo"""

    def __init__(self):
        self.name = 'mohan'
        self.age = 40
        self.marks = 80

    def talk(self):
        print("Hello! I am:", self.name)
        print("My Age is:", self.age)
        print("My Marks are:", self.marks)
```

What is an Object?

- An **object** is an instance of a class.
- Objects have physical existence in memory once created.
- We can create many different objects from the same class.

Syntax to Create Object:

```
reference_var = ClassName()
```

Example:

```
s = Student()
```

What is a Reference Variable?

- A **reference variable** refers to an object created from a class.
- It is used to access the object's attributes and methods.

Example:

```
s.talk()
```

This accesses the `talk()` method of the object `s`.

Class with Constructor Parameters

Program: Student Details with Constructor Parameters

```python
class Student:
    def __init__(self, name, rollno, marks):
        self.name = name
        self.rollno = rollno
        self.marks = marks

    def talk(self):
        print("Hello My Name is:", self.name)
        print("My Rollno is:", self.rollno)
        print("My Marks are:", self.marks)

s1 = Student("mohanlal", 101, 80)
s1.talk()
```

Output of Program

Output:

```
Hello My Name is: mohanlal
My Rollno is: 101
My Marks are: 80
```

This shows how the constructor initializes attributes and how method `talk()` displays them.

Types of Variables in Python Class

- Inside a Python class, we can define 3 types of variables:

1. **Instance Variables** (Object Level Variables)
2. **Static Variables** (Class Level Variables)
3. **Local Variables** (Method Level Variables)

1) Instance Variables

- Instance variables are those whose value varies from object to object.
- Each object gets its own copy of instance variables.

Where can we declare Instance Variables?

1. Inside constructor using `self` variable
2. Inside instance method using `self` variable
3. Outside class using object reference

Inside Constructor using self

We can declare instance variables inside the constructor using the `self` keyword. These variables get automatically added to the object.

```
class Employee:
    def __init__(self):
        self.eno = 100
        self.ename = 'Mohanlal'
        self.esal = 10000

e = Employee()
print(e.__dict__)
```

Output: `{'eno': 100, 'ename': 'Mohanlal', 'esal': 10000}`

Inside Instance Method using `self`

We can declare instance variables inside instance methods. These are created only when the method is explicitly called.

```
class Test:
    def __init__(self):
        self.a = 10
        self.b = 20

    def m1(self):
        self.c = 30

t = Test()
t.m1()
print(t.__dict__)
```

Output: `{'a': 10, 'b': 20, 'c': 30}`

Outside of Class using Object Reference

We can also add instance variables outside the class using object reference.

```
class Test:
    def __init__(self):
        self.a = 10
        self.b = 20

    def m1(self):
        self.c = 30

t = Test()
t.m1()
t.d = 40
print(t.__dict__)
```

Output: `{'a': 10, 'b': 20, 'c': 30, 'd': 40}`

Accessing Instance Variables

Inside a Python class, we can access instance variables:

- Using the `self` keyword within class methods
- Using object reference outside the class

Example:

```python
class Test:
    def __init__(self):
        self.a = 10
        self.b = 20
    def display(self):
        print(self.a)
        print(self.b)
t = Test()
t.display()
print(t.a, t.b)
```

```
10
20
10 20
```

2) Static Variables:

- If the value of a variable is not varied from object to object, such variables are called **static variables**.
- They are declared inside the class but outside any methods.
- Only **one copy** is created and **shared by all objects**.
- Static variables can be accessed using class name or object reference (class name is recommended).

Instance Variable vs Static Variable

Note:

- **Instance Variable:** Every object has its own copy.
- **Static Variable:** One copy shared by all objects of the class.

```
class Test:
    x = 10
    def __init__(self):
        self.y = 20

t1 = Test()
t2 = Test()
print('t1:', t1.x, t1.y)
print('t2:', t2.x, t2.y)
Test.x = 888
t1.y = 999
print('t1:', t1.x, t1.y)
print('t2:', t2.x, t2.y)
```

Various Places to Declare Static Variables

1. Inside class but outside methods
2. Inside constructor using class name
3. Inside instance method using class name
4. Inside class method using class name or cls
5. Inside static method using class name

Various Places to Declare Static Variables

```
class Test:
    a = 10
    def __init__(self):
        Test.b = 20
    def m1(self):
        Test.c = 30
    @classmethod
    def m2(cls):
        cls.d1 = 40
        Test.d2 = 400
    @staticmethod
    def m3():
        Test.e = 50
```

Static Variable Dictionary View

```
print(Test.__dict__)
t = Test()
print(Test.__dict__)
t.m1()
print(Test.__dict__)
Test.m2()
print(Test.__dict__)
Test.m3()
print(Test.__dict__)
Test.f = 60
print(Test.__dict__)
```

How to Access Static Variables

1. In constructor: using `self` or class name
2. In instance method: using `self` or class name
3. In class method: using `cls` or class name
4. In static method: using class name
5. Outside class: using object or class name

How to Access Static Variables

```
class Test:
    a = 10
    def __init__(self):
        print(self.a)
        print(Test.a)
    def m1(self):
        print(self.a)
        print(Test.a)
    @classmethod
    def m2(cls):
        print(cls.a)
        print(Test.a)
    @staticmethod
    def m3():
        print(Test.a)
```

Accessing Static Variables via Object or Class

```
t = Test()
print(Test.a)
print(t.a)
t.m1()
t.m2()
t.m3()
```

Where We Can Modify Static Variables

Anywhere either within the class or outside the class using class name.

Inside class method: use `cls` variable.

```
class Test:
    a = 777
    @classmethod
    def m1(cls):
        cls.a = 888
    @staticmethod
    def m2():
        Test.a = 999

print(Test.a)   # 777
Test.m1()
print(Test.a)   # 888
Test.m2()
print(Test.a)   # 999
```

3) Local Variables:

- Sometimes to meet temporary requirements of programmer, we can declare variables inside a method directly.
- Such variables are called **local variables** or **temporary variables**.
- Local variables will be created at the time of method execution and destroyed once the method completes.
- **Important:** Local variables of a method cannot be accessed from outside of that method.

Example of Local Variables (Valid Access)

```python
class Test:
    def m1(self):
        a = 1000
        print(a)

    def m2(self):
        b = 2000
        print(b)

t = Test()
t.m1()
t.m2()
```

Output:

- 1000
- 2000

Example: Accessing Local Variable Outside Its Scope (Error)

```python
class Test:
    def m1(self):
        a = 1000
        print(a)
    def m2(self):
        b = 2000
        print(a)       # NameError: name 'a' is not defined
        print(b)
t = Test()
t.m1()
t.m2()
```

Output:

- 1000
- NameError: name 'a' is not defined

Types of Methods in Python

Python classes support three types of methods:

1. **Instance Methods**
2. **Class Methods**
3. **Static Methods**

Each has different access and use depending on whether they access instance, class, or no internal data.

Instance Methods

- Use instance variables via `self`.
- Automatically receives the object as the first argument (`self`).
- Accessed via object reference.

Instance Methods

```python
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks
    def display(self):
        print('Hi', self.name)
        print('Your Marks are:', self.marks)
    def grade(self):
        if self.marks >= 60:
            print('You got First Grade')
        elif self.marks >= 50:
            print('You got Second Grade')
        elif self.marks >= 35:
            print('You got Third Grade')
        else:
            print('You are Failed')
```

Instance Methods

```python
n = int(input('Enter number of students:'))
for i in range(n):
    name = input('Enter Name:')
    marks = int(input('Enter Marks:'))
    s = Student(name, marks)
    s.display()
    s.grade()
    print()
```

Class Methods

- Work with class-level data (shared across all instances).
- First argument is `cls`, representing the class itself.
- Defined using `@classmethod` decorator.

```python
class Animal:
    legs = 4

    @classmethod
    def walk(cls, name):
        print('{} walks with {} legs...'.format(name, cls.legs))

Animal.walk('Dog')
Animal.walk('Cat')
```

Output:

```
Dog walks with 4 legs...
Cat walks with 4 legs...
```

Class Method Example: Count Objects

Track number of objects created for a class

```
class Test:
    count = 0

    def __init__(self):
        Test.count += 1

    @classmethod
    def noOfObjects(cls):
        print('The number of objects created for test class:',
            cls.count)
```

Class Method Example: Count Objects

```

t1 = Test()
t2 = Test()
Test.noOfObjects()
t3 = Test()
t4 = Test()
t5 = Test()
Test.noOfObjects()
```

Static Methods

- Do not access instance or class data.
- No `self` or `cls` passed as arguments.
- Use `@staticmethod` decorator.

```python
class Math:
    @staticmethod
    def add(x, y):
        print('The Sum:', x + y)
    @staticmethod
    def product(x, y):
        print('The Product:', x * y)
    @staticmethod
    def average(x, y):
        print('The average:', (x + y) / 2)
Math.add(10, 20)
Math.product(10, 20)
Math.average(10, 20)
```

Notes on Method Types

- Instance methods are the most common; used to access object state.
- Static methods are for utility operations unrelated to class/object.
- Class methods manipulate class-level data; rarely used.

Summary:

Method Type	First Arg	Use Case
Instance Method	self	Access instance variables
Class Method	cls	Access class variables
Static Method	none	General utility functions

Thank You!

Questions are welcome.